TALLER 2: REDES NEURONALES

Elkin Daniel Ramos Rodríguez
78158
Valentina Zarate Perdomo
114109

Universidad ECCI

DOCENTE
Fredy Alexander Orjuela López

SISTEMAS AVANZADOS DE PRODUCCIÓN
INGENIERIA INDUSTRIAL

12 DE ABRIL DE 2026
BOGOTA, COLOMBIA

Contenido

Introducción 2

Estructura de evaluación 3

Desarrollo analítico Regularización Ridge 3

Representación vectorial 3

Expansión de la forma cuadrática 4

Condición de primer orden 5

Solución cerrada 6

Sobre la invertibilidad 7

Efecto de parámetro 7

Optimización numérica y descenso de gradiente 8

Impacto de la Tasa de Aprendizaje 8

Dependencia del punto inicial 10

Fenómeno de "Overshooting" 11

Eficiencia Computacional 12

Generalización 13

Introducción

El análisis de datos se ha convertido en una herramienta clave en el ámbito de la ingeniería industrial y de las ciudades inteligentes, así como en la optimización de sistemas y en la toma de decisiones. En particular, los sistemas de bicicletas compartidas son un claro ejemplo de cómo un alto volumen de datos permite modelizar la conducta de la demanda a partir de variaciones temporales y de otros aspectos externos, como el clima. Este taller tiene como finalidad aplicar técnicas analíticas avanzadas para abordar y modelizar la interpretación de patrones o comportamientos complejos, que van desde modelos lineales regularizados hasta redes neuronales. A lo largo del curso del taller, se abordan conceptos, como la regresión Ridge, el descenso a lo largo de la pendiente, la optimización de los hiperparámetros o la propagación adelante hacia en las redes neuronales, que se presentan como fundamentos matemáticos junto a su posible implementación a través de la computación. Se persigue, además, analizar el rendimiento de los modelos, no solo desde la perspectiva de una evaluación estadística, sino también mediante la

Evaluación ante situaciones reales, tales como la logística del transporte o el mantenimiento predictivo en sistemas industriales. De esta forma, el taller permite fortalecer competencias en los ámbitos del modelado, de la interpretación de resultados y de la toma de decisiones en la base de datos, todo lo que, por su parte, resulta fundamental para el ejercicio profesional en ingeniería.

Estructura de evaluación

Desarrollo analítico Regularización Ridge

En escenarios de alta dimensionalidad o cuando existe una fuerte correlación entre los predictores (como podría ocurrir entre la inversión en TV y Radio), el estimador de Mínimos Cuadrados Ordinarios (OLS) tiende a presentar una varianza excesiva. Para mitigar esto, se utiliza la Regresión Ridge, la cual añade una penalización a la magnitud de los coeficientes.

La función de costo a minimizar es la siguiente:

$$J(\beta) = \sum_{i=1}^{n} \left(\mathrm{yi} - \beta 0 - \sum_{j=1}^{p} \beta j * Xij \right)^2 + \gimel \sum_{j=1}^{p} \beta j^2$$

Representación vectorial

Los valores observados de la variable que se agrupan en un vector columna

$$y = \begin{pmatrix} y1 \\ y2 \\ y3 \\ . \\ yn \end{pmatrix}$$

Definimos la matriz de diseño X contiene todas las variables predictoras el modelo tiene intercepto

$$x = \begin{pmatrix} 1 & X11 & X12 & \cdots & X1p \\ 1 & X21 & X22 & \cdots & X2p \\ 1 & X31 & X32 & \cdots & X3p \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & Xn1 & Xn2 & \cdots & Xnp \end{pmatrix}$$

Vector de parámetro β

$$\beta = \begin{pmatrix} \beta 0 \\ \beta 1 \\ \beta 2 \\ \vdots \\ \beta p \end{pmatrix}$$

La predicción del modelo lineal y se escribe $X\beta$ de tal manera que se produce un vector columna con todas las predicciones

$$X\beta = \begin{pmatrix} \beta 0 + \beta 1X11 + \beta 2X12 + \cdots + \beta pX1p \\ \beta 0 + \beta 1X21 + \beta 2X22 + \cdots + \beta pX2P \\ \vdots \\ \beta 0 + \beta 1Xn1 + \beta 2Xn2 + \cdots + \beta pXnp \end{pmatrix}$$

Valor vector de errores este valor nos ayuda a la diferencia entre el valor real y el valor predicho:

$$Y - X\beta$$

Suma de cuadrados de los errores

$$\sum_{i=1}^{n} \left(Yi - \beta 0 - \sum_{j=1}^{p} \beta jXij \right)^2$$

Una manera matricial, esta suma se puede expresar:

$$(Y - X\beta)^T (Y - X\beta)$$

Penalización Ridge en forma vectorial

$$\lambda \sum_{j=1}^{p} \beta j^2$$

Suma de cuadrados de los coeficientes

$$\beta^T \beta$$

La penalización queda

$$\lambda \beta^T \beta$$

Función completa en forma vectorial

$$J(\beta) = (Y - X\beta)^T (Y - X\beta) + \lambda \beta^T \beta$$

Expansión de la forma cuadrática

Expandir el producto $(Y - X\beta)^T (Y - X\beta)$

Propiedad distributiva

$$(Y - X\beta)^T(Y - X\beta) = Y^T(Y - X\beta) - (X\beta)^T(Y - X\beta)$$

$$Y^TY - Y^TX\beta - (X\beta)^TY + (X\beta)^TX\beta$$

Simplificar termino

$$(X\beta)^T = \beta^TX^T$$

$$(X\beta)^TY = \beta^TX^TY$$

$$(X\beta)^TX\beta = \beta^TX^TY\beta$$

Entonces

$$(Y - X\beta)^T(Y - X\beta) = Y^TY - Y^TX\beta - \beta^TX^TY + \beta^TX^TX\beta$$

Identificación de términos iguales

$$Y^TX\beta \textbf{ Y } \beta^TX^TY$$

$$Y^TX\beta = \beta^TX^TY$$

$$(Y - X\beta)^T(Y - X\beta) = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta$$

Termino de penalización

$$\measuredangle \beta^t\beta$$

$$J(\beta) = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta + \lambda\beta^T\beta$$

Términos semejantes

$$J(\beta) = Y^TY - 2\beta^TX^TY + \beta^T(X^TX + \lambda I)\beta$$

Condición de primer orden

$$J(\beta) = Y^TY - 2\beta^TX^TY + \beta^T(X^TX + \lambda I)\beta$$

$$\frac{\partial J(\beta)}{\partial \beta}$$

Derivar primer termino

$$Y^TY$$

$$\frac{\partial(Y^TY)}{\partial \beta} = 0$$

Derivar segundo termino

$$-2\beta^TX^TY$$

$$\frac{\partial(\beta^T\alpha)}{\partial\beta} = \alpha$$

$$\alpha = X^TY$$

$$\frac{\partial(-2\beta^TX^TY)}{\partial\beta} = -2X^TY$$

Derivar tercer termino

$$\beta^t(X^TX + \lambda I)\beta$$

$$\frac{\partial(\beta^TA\beta)}{\partial\beta} = 2A\beta$$

$$A = X^TX + \lambda I$$

$$\frac{\partial(\beta^T(X^TX + \lambda I)\beta)}{\partial\beta} = 2(X^TX + \lambda I)\beta$$

Gradiente completo

$$\frac{\partial J(\beta)}{\partial\beta} = 0 - 2X^TY + 2(X^TX + \lambda I)\,\beta$$

$$\frac{\partial J(\beta)}{\partial\beta} = -2X^TY + 2(X^TX + \lambda I)\beta$$

Gradiente el vector nulo

$$\frac{\partial j(\beta)}{\partial\beta} = 0$$

$$-2X^TY + 2(X^TX + \lambda I)\beta = 0$$

$$-X^TY + (X^TX + \lambda I)\beta = 0$$

$$(X^TX + \lambda I)\beta = X^TY$$

Resultado condición de primer orden

$$(x^TX + \lambda I)\beta = X^TY$$

Solución cerrada

$$(x^TX + \lambda I)\beta = X^TY$$

Ecuación obtenida

$$(X^TX + \lambda I)\beta = X^TY$$

$$X^TX + \lambda I$$

Multiplicación de inversa de la matriz $(x^tx + \lambda i)$

$$(X^TX + \lambda I)^{-1}(X^TX + \lambda I)\beta = (X^TX + \lambda I)^{-1}X^TY$$

Simplificar el lado izquierdo

$$(X^TX + \lambda I)^{-1}(X^TX + \lambda I) = I$$

$$I\beta = (X^TX + \lambda I)^{-1}X^TY$$

$$\beta = (X^TX + \lambda I)^{-1}X^TY$$

Estimador ridge

$$\hat{\beta}ridge = (X^TX + \lambda I)^{-1}X^TY$$

Sobre la invertibilidad

La suma de λI garantiza la invertibilidad de la matriz $(X^TX + \lambda I)$ porque desplaza sus valores propios positivos. Cuando X^TX es singular, algunos de sus valores propios son cero, esto hace que impide calcular su inversa. Sin embargo, al añadir λI (con $\lambda > 0$), todos estos valores propios aumentan en λ, volviéndose estrictamente positivos. Como resultado, la matriz se vuelve definida positiva y, por tanto, invertible, lo que permite obtener una solución estable en la regresión ridge.

Efecto de parámetro

Cuando $\lambda \to 0$, La restricción de coeficientes desaparece y el modelo Ridge se comporta como una regresión de mínimos cuadrados ordinarios, lo que da lugar a coeficientes sin restricción, bajo sesgo, elevada varianza y un alto riesgo de sobreajuste. Por el contrario, cuando $\lambda \to \infty$, la penalización predomina, y los coeficientes tienden a ser cero emitiendo un modelo excesivamente simplificado, lo que genera mucho sesgo y poca varianza, provocando por tanto subajuste. De este modo, λ es un parámetro que controla la complejidad del modelo, de modo que le permite encontrar un buen equilibrio entre sesgo y varianza de forma que se mejora la capacidad de generalización.

Optimización numérica y descenso de gradiente

En esta fase, exploraremos el "motor" que permite entrenar modelos complejos como las Redes Neuronales: el algoritmo de Descenso de Gradiente. El objetivo es entender cómo la configuración de los hiperparámetros determina si un modelo logra converger al mínimo global de la función de pérdida o si falla en el intento.

Impacto de la Tasa de Aprendizaje

```python
    for i in range(num_iterations):
        z = f(point[0], point[1])
        path.append([point[0], point[1], z])

        # Calcula el gradiente
        grad = grad_f(point[0], point[1])

        # Actualiza la posición del punto
        point = point - learning_rate * grad

    return np.array(path)

# --- CONFIGURACIÓN DE LA SIMULACIÓN ---
# Punto de inicio (puedes cambiarlo para ver cómo cae en otro mínimo)
start_point = [0.0, 1.0]
learning_rate = 0.0001       # Tasa de aprendizaje
num_iterations = 1000        # Número de pasos

# Ejecutar el algoritmo
path = gradient_descent(start_point, learning_rate, num_iterations)

# --- PREPARACIÓN DE LA VISUALIZACIÓN ---
# Crear la malla de puntos para la superficie 3D y las curvas de nivel
x = np.linspace(-7, 7, 200)
y = np.linspace(-7, 7, 200)
X, Y = np.meshgrid(x, y)
Z = f(X, Y)
```

Ilustración 1 impacto tasa de aprendizaje

¿Qué sucede con su trayectoria?

Al aumentar la tasa de aprendizaje a unos valores altos como $\eta=0.5$ o $\eta=1.0$, la trayectoria del descenso del gradiente pasa a ser aventurada y poco precisa, en los dos casos, el algoritmo logra acercarse a la zona del mínimo, pero no va a parar precisamente en el centro de las curvas de nivel, sino que se comporta de forma errática, ya que los pasos son de gran tamaño y, con ello, el algoritmo se pasa ligeramente en cada iteración del mínimo.

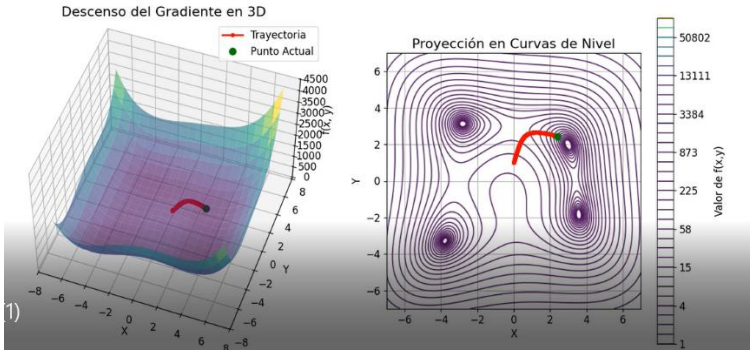


Ilustración 2 n= (1.0)

La trayectoria con η=0.5 es relativamente estable, aunque imprecisa en la convergencia final; en cambio, con η=1.0 los movimientos son más bruscos, por lo que se acaba generando mayor inestabilidad y menor precisión a la hora de converger hacia el mínimo.

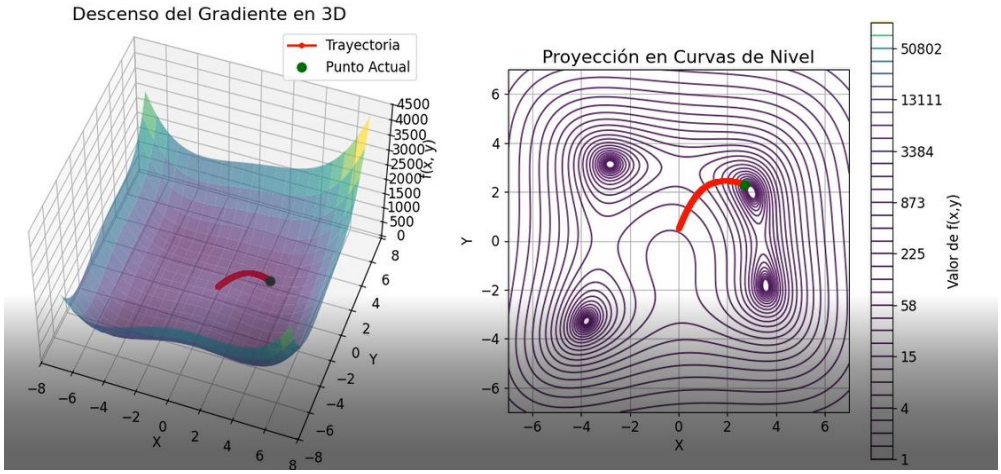


Ilustración 3 n= (0.5)

En conclusión, el algoritmo no diverge en estos casos, pero tampoco se asegura una convergencia óptima, por lo que se puede concluir que altas tasas de aprendizaje afectan negativamente y de manera severa a la estabilidad y precisión del descenso del gradiente.

¿Es suficiente este tiempo de cómputo para alcanzar el mínimo?

Si la tasa de aprendizaje es un número extremadamente pequeño, digamos $\eta = 10^{-7}$, entonces el algoritmo toma pasos en cada iteración. En este caso, aunque la dirección del movimiento es la correcta se está moviendo a la dirección del mínimo después de 1000 iteraciones los cambios en la posición son tan insuficientes que la posición todavía queda alejada del mínimo. Por otro lado, si el orden de los pasos en cada iteración es excesivamente pequeño, entonces el tiempo de cómputo no basta para encontrar la solución, pues el algoritmo sí converge, pero lo hace lentamente e ineficazmente.

Dependencia del punto inicial

Al comprobar el hecho de que modificando el start_point a coordenadas alejadas del origen, se ha observado que el algoritmo no necesariamente se ve conducido siempre, de nuevo, hacia la misma solución final. Como se puede ver, en las imágenes las trayectorias terminan en regiones diferentes de la superficie, lo que indica que el descenso del gradiente convergería hacia diferentes mínimos dependiendo de la posición inicial elegida.

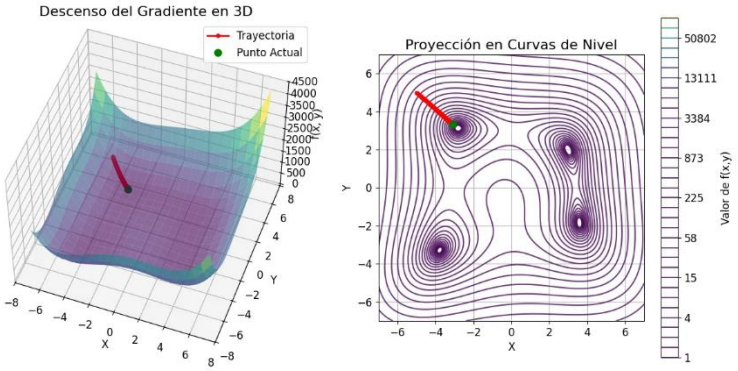


Ilustración 4 punto inicial (-5.0,5.0)

Esto se debe a que la función f (X, Y) tiene una topografía de la superficie con diferentes valles o mínimos locales. A partir de la gráfica de curvas de nivel se puede distinguir un número de zonas cerradas que se separan entre sí y que funcionarían como "cuencas de atracción" y esto explica el hecho de que cuando el algoritmo se inicia desde lugares cercanos a una de esas zonas cerradas el algoritmo se internaliza para el mínimo más cercano y no tiende a converger hacia un único punto global.

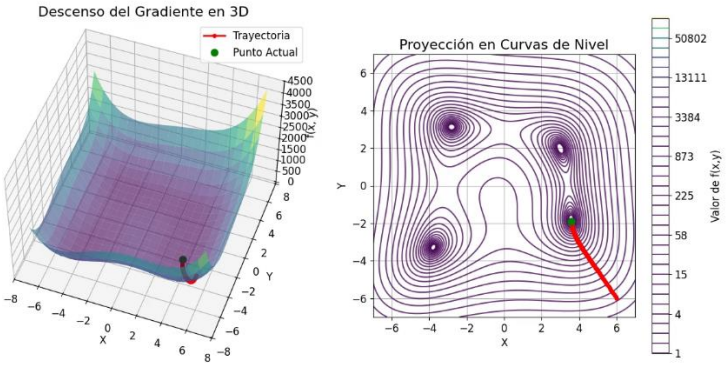


Ilustración 5 punto inicial (6.0, -6.0)

Así pues, el punto de inicio tiene un efecto directo sobre el resultado final que obtendrá el algoritmo. Este hecho pone de manifiesto que en funciones no convexas como en este caso el descenso del gradiente puede conducir a diferentes resultados dependiendo de donde comience a buscar.

Fenómeno de "Overshooting"

El overshooting se produce cuando el tamaño del paso ($\eta \cdot \nabla f$) es más grande que la distancia al mínimo. Esto quiere decir que, en lugar de aproximarse lentamente a la solución buscada, el algoritmo la sobresaldrá y caerá por el otro lado de la superficie. Es así como el algoritmo comienza

una oscilación eterna en vez de una convergencia; esta oscilación aparece como un movimiento continuo alrededor del mínimo.

Como se puede observar en el gráfico de las curvas de nivel, se trata de una trayectoria que continuamente pasa por el centro de las curvas de nivel, pasando de un lado a otro sin quedar fijada en su posición. En lugar de una aproximación gradual, los puntos de las curvas de nivel presentan saltos gigantescos, hasta el punto que puede apreciarse que se trata de un tamaño de paso de una magnitud inadecuada.

Eficiencia Computacional

Si logramos que el algoritmo se detenga en la iteración 1000 pero está todavía muy lejos del mínimo, la mejor opción que nos sería aconsejable seguir es modificar la tasa de aprendizaje y no simplemente aumentar el número de iteraciones; ello es debido a que tener una tasa de aprendizaje muy pequeña nos obligará a hacer pequeños pasos, en cuyo caso, y ya que hay muy poco avance en las iteraciones, el número de iteraciones queda trucado porque acaba prolongando un proceso ineficiente.

Aumentando adecuadamente la tasa de aprendizaje del algoritmo, se logrará una mejor computación en la dirección del mínimo. En consecuencia, sería mejor encontrar un valor óptimo de η que nos permita avanzar en el proceso y que a su vez nos garantice estabilidad que no quedarnos sólo a expensas del número de iteraciones.

Generalización

En los casos de ingeniería en los que hay muchas variables, asumir una única posición inicial aleatoria es poco prudente, ya que el algoritmo puede converger a diferentes mínimos locales según en qué posición se haya comenzado a trabajar. Esto se manifestaba en los resultados de las simulaciones, donde una misma alternativa daba como fruto soluciones distintas: no había una única solución posible.

Esto puede llevar a decisiones subóptimas, especialmente en situaciones del tipo optimización de cadenas de suministro, donde existe la posibilidad de que alguna solución local no corresponda a la mejor solución global. Por eso, se encuentra que conviene la utilización de muchas posiciones iniciales, de técnicas de validación o de métodos más robustos, que reduzcan la dependencia de la posición inicial.

Redes neuronales (MLP) y ajuste de hiperparámetros

Fase 3a

Diferenciación de Métricas (MAE vs RMSE)

El **MAE (Error Absoluto Medio)** mide el promedio de los errores en valor absoluto, es decir, todas las diferencias entre valores reales y predichos tienen el mismo peso.

Por otro lado, el **RMSE (Raíz del Error Cuadrático Medio)** eleva los errores al cuadrado antes de promediarlos, lo que hace que los errores grandes tengan un impacto mucho mayor en el resultado final.

Diagnóstico de Desempeño (R^2 train vs test)

Si el valor de R2R^2R2 en entrenamiento es significativamente mayor que en el conjunto de prueba, esto indica un problema de **sobreajuste (overfitting)**.

Esto significa que:

- El modelo aprende muy bien los datos de entrenamiento
- Pero **no generaliza bien a datos nuevos**

Una solución es aplicar **regularización**, como en la regresión Ridge, lo que equivale en redes neuronales a:

- Aumentar el parámetro alpha
- Reducir la complejidad del modelo (menos neuronas/capas)
- Mantener o mejorar early_stopping

Esto reduce la varianza del modelo y mejora su capacidad de generalización.

Interpretación de residuos

Con base a las gráficas percibimos que los puntos siguen la diagonal el modelo logra capturar la tendencia general, pero en valores altos los puntos tienden a quedar pro debajo de la diagonal lo que podemos concluir que esta centrado cerca de cero y tiene una dispersión lo que podemos concluir que tiene un buen comportamiento, pero presenta errores importantes en algunos casos.

Fase 3ª parte 2

```python
# ==========================================================
# MÉTODO 2: MLPRegressor (Red neuronal sencilla para regresión)
# ----------------------------------------------------------
# Qué hace esta celda:
# 1) Entrena un MLP con 1-2 capas pequeñas (no lineal)
# 2) Usa early_stopping para evitar sobreajuste
# 3) Evalúa MAE, RMSE, R2 en train y valid
# 4) Grafica y_real vs y_pred, residuales y curva de pérdida
# ==========================================================

import numpy as np
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

mlp = MLPRegressor(
    hidden_layer_sizes=(32,16, 8,4),  # empieza simple; puedes ajustar
    activation="relu",
    solver="adam",
    learning_rate_init=1e-5,
    max_iter=200,
    early_stopping=True,      # usa un conjunto interno de validación
    validation_fraction=0.2,  # 20% del conjunto de entrenamiento
    n_iter_no_change=10,
    random_state=42
)
mlp.fit(X_train_pre, y_train)
```

Arquitectura

La arquitectura especifica el número de capas ya las neuronas que tendrá la red.

Por ejemplo, en el caso de las zonas profundas: (32,16,8,4) contienen más capas y mayor capacidad de aprender de relaciones complejas y no lineales. En el caso de las redes muy anchas: (64,) que concentraría más neuronas en una sola capa podría conseguir aprender patrones relevantes, pero con menor profundidad estructural.

Interpretación

Si la red es excesivamente compleja se corre el riesgo de que se pueda producir overfitting (si lo que está haciendo es "memorizar" los datos). En cambio, si es muy simple podría incurrirse en underfitting (si no es capaz de modelar bien la relación entre variables).

Redacción lista para el informe

Comparar arquitecturas profundas frente a anchas ayuda a calibrar equilibrio entre capacidad de aprender respecto a complejidad del modelo. En general, las redes profundas suelen captar patrones no lineales más elaborados, en cambio las redes anchas ofrecen una estructura

mucho más sencilla. La selección óptima dependerá de qué arquitectura sea capaz de alcanzar mejor generalización en validación-test.

Regularización

El parámetro alpha de MLPRegressor es la penalización L2 por los pesos de la red, análoga a la regularización Ridge que habíamos visto en la Fase 1.

alpha pequeño: la penalización es débil, la red es más flexible, puede aprender más, pero aumenta el riesgo de sobredimensionar (overfitting).

alpha grande: la penalización es fuerte, los pesos se reducen, la red se vuelve rígida y se corre el riesgo de subajustar (underfitting).

¿Cómo afecta un alpha muy elevado?

Un α muy grande limita mucho la capacidad de la red, pues fuerza a que los pesos sean muy pequeños. Esto reduce la complejidad del modelo; sin embargo, aprenderá patrones que deberían ser aprendidos por la red.

Redacción lista para el informe

El parámetro alpha controla la regularización del modelo y asume un efecto análogo a la penalización Ridge. Valores pequeños permiten mayor capacidad de aprendizaje, en cambio, valores altos restringen en exceso los pesos y la red pierde la posibilidad de aprender, haciendo que un α muy elevado lleve al subajuste de la red, y por tanto que la red se avenga a ser una red neuronal demasiado simplificada.

Tasa de aprendizaje

Este parámetro controla el tamaño de los pasos en la actualización de los pesos del modelo.

Tasa grande: el entrenamiento avanza más velozmente, pero puede ser inestable, mostrando a lot of overshooting.

Tasa baja: el aprendizaje es también más estable, pero es más lento. Relación con Early Stopping

El early_stopping para el entrenamiento cuando el error de validación deja de mejorar.

Si la tasa de aprendizaje es muy grande, el modelo puede oscilar una barbaridad y el early_stopping se activa enseguida, antes de haber convergido bien.

Si la tasa es muy baja, el aprendizaje puede ser muy lento y el modelo puede necesitar muchas iteraciones para llegar a un aprendizaje significativo.

Fase 3b

Consistencia de parámetros

Así es, existe una relación directa entre la complejidad de la red y la necesidad de regularización (alpha).

Cuando la red neuronal es más compleja (más capas o más neuronas) posee mayor capacidad de aprendizaje, pero también mayor riesgo de sobreajuste. Así, será necesario establecer un valor de alpha más alto para que se penalicen los pesos y evitar que el modelo se ajuste más allá de lo necesario con respecto a los datos de entrenamiento.

Esto es congruente con lo visto en la Regresión Ridge, donde la regularización permite controlar los coeficientes para reducir la varianza del modelo.

Early Stopping

El early_stopping es un mecanismo que para el entrenamiento cuando el modelo deja de mejorar en el conjunto de validación.

¿En qué me ayuda?

Reduce tiempo de cómputo: deja de entrenar cuando ya no hay mejoras; Previene débil ajuste: corta el entrenamiento antes de que el modelo aprenda ruido.

Primero la pérdida baja rápidamente, después de su estabilización lo que significa que el simulador aprendió lo que necesitaba y que de seguir entrenando no va a generar mejoras.

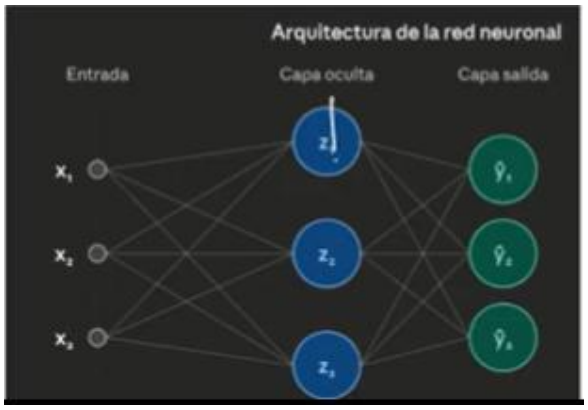
Conclusión de Negocio

El modelo es adecuado para obtener estimaciones orientativas y ayudar en decisiones de relevancia media porque logra capturar la tendencia principal que admiten las ventas. Sin embargo, no resulta enteramente robusto para decisiones realmente cruciales, especialmente en un escenario en el que las ventas puedan marcarse con alta demanda, ya que presenta una tendencia a subestimar altos valores en pro de profundas imprecisiones en los extremos.

Por este motivo, se sugiere usar el modelo como consuelo, ya que la ubicación en los resultados quizás se vea mejorada por análisis adicionales u optimizaciones de sus hiperparámetros.

3.4. Fase 4: Propagación hacia Adelante (Forward Propagation) Manual

$$x: \begin{pmatrix} 0{,}5 \\ 0{,}3 \\ 0{,}2 \end{pmatrix}$$



$$w^1: \begin{pmatrix} w_{11} & w_{12} & w_{13} \\ w_{21} & w_{22} & w_{23} \\ w_{31} & w_{32} & w_{33} \end{pmatrix}$$

$$z_1 = 0{,}2\ x\ 0{,}5 + 0{,}5\ x\ 0{,}3 + (-0{,}1) x\ 0{,}2 + 0{,}1 = \ 0{,}33$$

1. Puntaje de la Capa Oculta 0,33 / 0,19 / -0,18

0,2	0,5	-0,1	0,5
0,8	-0,3	0,4	0,3
-0,5	0,1	0,2	0,2
0,23	0,1	Z1	0,33
0,39	-0,2	Z2	0,19
-0,18	0	Z3	-0,18
Sigmoide	Entra a la capa 2		
0,58175938			
0,54735762			
0,45512111			

2. Activación de la Capa Oculta: $\sigma(z)\frac{1}{1+e^{-z}}$

- 0,58175938
- 0,54735762
- 0,45512111

0,5	-0,4	0,1	0,58175938
0,2	0,8	-0,5	0,54735762
-0,1	0,3	0,7	0,45512111
0,11744875	-0,1	Salida 1	0,01744875
0,32667742	0,2	Salida 2	0,52667742
0,42461612	0,1	Salida 3	0,52461612
SOFTMAX			
0,23123591	23,12359052	Clase 1	
0,38477821	38,47782092	Clase 2	
0,38398589	38,39858855	Clase 3	
1			

https://docs.google.com/spreadsheets/d/1LiB_q2WCLGpuJJN5vOmcGOKcIO6EyHaL/edit?usp=sharing&ouid=103995350744161519690&rtpof=true&sd=true

3. Puntaje de la Capa de Salida (Z (2)): Calcule Z (2) = W (2)A(1) + b (2).

Resultados salida

- 0,01744875
- 0,52667742
- 0,52461612

4.

SOFTMAX	Probabilidad
0,23123591	23,12359052
0,38477821	38,47782092
0,38398589	38,39858855

Interpretación de Resultados: Según las probabilidades obtenidas, ¿en qué categoría clasifica la red a esta campaña publicitaria? **RPTA:** De acuerdo con las probabilidades obtenidas mediante la función Softmax, la mayor probabilidad corresponde a la clase es P2: 38,48.

Por lo tanto, la red neuronal clasifica la campaña como Impacto medio (C2), Esto indica que el modelo considera que la combinación de inversión en TV, radio y periódico genera un impacto moderado en el desempeño de la campaña publicitaria

Función de Activación: ¿Por qué utilizamos Softmax en la última capa en lugar de una

función lineal o una sigmoide simple? Explique en términos de la suma de las probabilidades. **RPTA:** La función Softmax se utiliza en la última capa porque transforma los valores de salida en probabilidades entre 0 y 1, cuya suma total es igual a 1.

En este caso:

$$0{,}2312 + 0{,}3848 + 0{,}3839 = 1$$

Esto permite interpretar fácilmente la salida del modelo como la probabilidad de pertenecer a cada clase.

A diferencia de una función lineal o sigmoide, Softmax evalúa todas las salidas simultáneamente, lo que la hace ideal para problemas de clasificación multiclase.

Rol del Bias: ¿Qué sucedería con la capacidad de representación de la red si todos los

vectores de sesgo (b) fueran cero? **RPTA:** El bias permite ajustar la salida de cada neurona independientemente de las entradas, funcionando como un desplazamiento

Si todos los bias fueran cero, la red perdería flexibilidad, ya que las funciones de activación estarían limitadas y pasarían por el origen

Esto reduciría la capacidad del modelo para aprender patrones complejos, afectando negativamente su precisión y desempeño

3.5. Fase 5: Evaluación de Modelos y Toma de Decisiones (Caso: Mantenimiento Predictivo)

```
# URL del dataset en el repositorio de UCI
url = '/content/ai4i2020.csv'

# Cargar los datos en un DataFrame de pandas
df = pd.read_csv(url)

# Mostrar las primeras 5 filas para verificar la carga
print("Datos cargados exitosamente. Primeras 5 filas:")
df.head()
```

Datos cargados exitosamente. Primeras 5 filas:

	UDI	Product ID	Type	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	TWF	HDF	PWF	OSF	RNF
0	1	M14860	M	298.1	308.6	1551	42.8	0	0	0	0	0	0	0
1	2	L47181	L	298.2	308.7	1408	46.3	3	0	0	0	0	0	0
2	3	L47182	L	298.1	308.5	1498	49.4	5	0	0	0	0	0	0
3	4	L47183	L	298.2	308.6	1433	39.5	7	0	0	0	0	0	0
4	5	L47184	L	298.2	308.7	1408	40.0	9	0	0	0	0	0	0

ANALISIS: El algoritmo intenta corregir el severo desbalance de clases detectado en el conjunto de datos. Según el análisis exploratorio (EDA), solo el 3.39% de las muestras corresponden a fallos reales (Clase 1), mientras que la gran mayoría representa un estado normal (Clase 0).

Compensación de la minoría: Al asignar un peso 14.7 veces mayor a la Clase 1, se obliga al modelo a no ignorar los pocos ejemplos de falla disponibles.

Ajuste del costo de error: En mantenimiento, un Falso Negativo (no detectar una falla) es mucho más costoso que un Falso Positivo. El parámetro class_weight penaliza con mayor fuerza al modelo si se equivoca omitiendo un fallo.

Frontera de decisión: Esto evita que el modelo aprenda una solución "perezosa" de predecir siempre "No Falla" para obtener una métrica alta de forma artificial.

EDA: Análisis de la Variable Objetivo (Desbalance de Clases)

Este es el paso más importante en un problema de clasificación: ver cómo están distribuidas las clases.

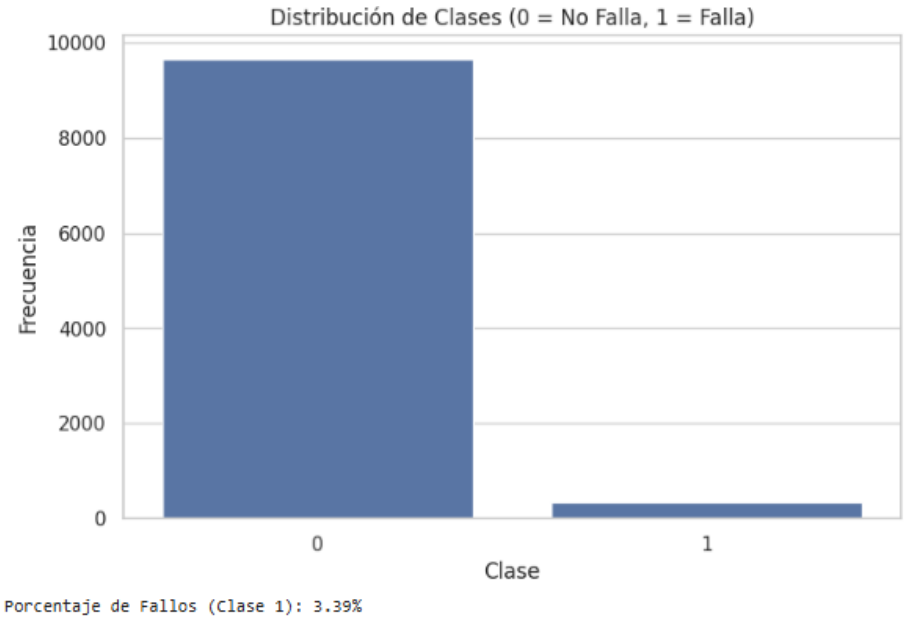


Figura 1. Distribución de clases y porcentaje de fallos.

ANALISIS: Valida el desbalance mencionado: la Clase 0 domina visualmente sobre la Clase 1 (3.39%)

Veamos si las variables de los sensores se comportan diferente cuando hay un fallo.

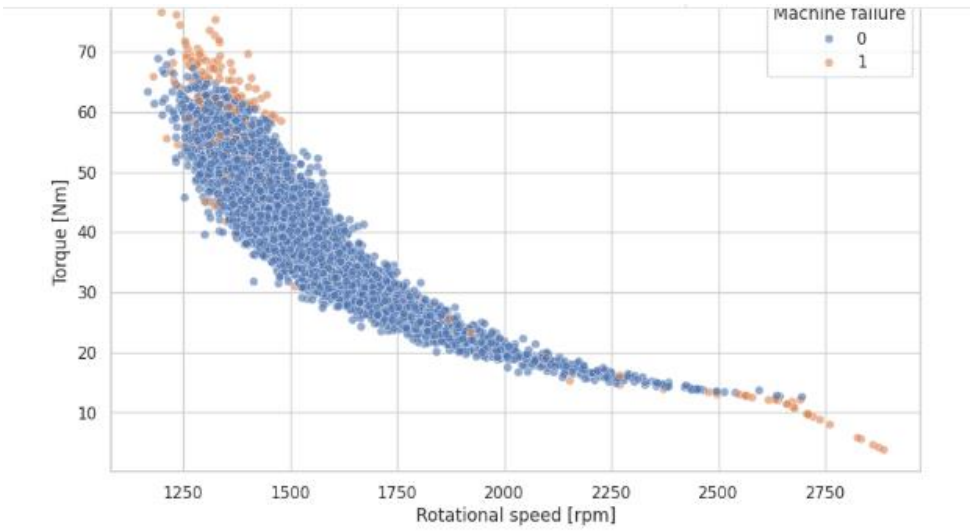
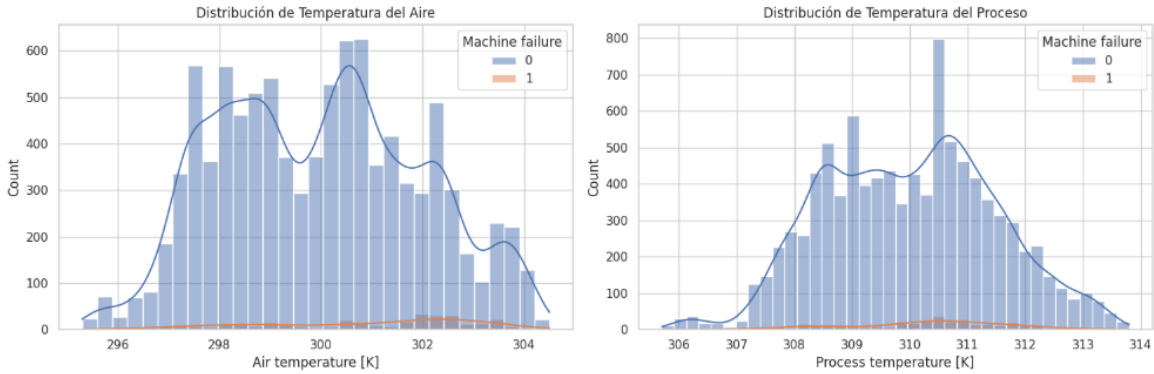
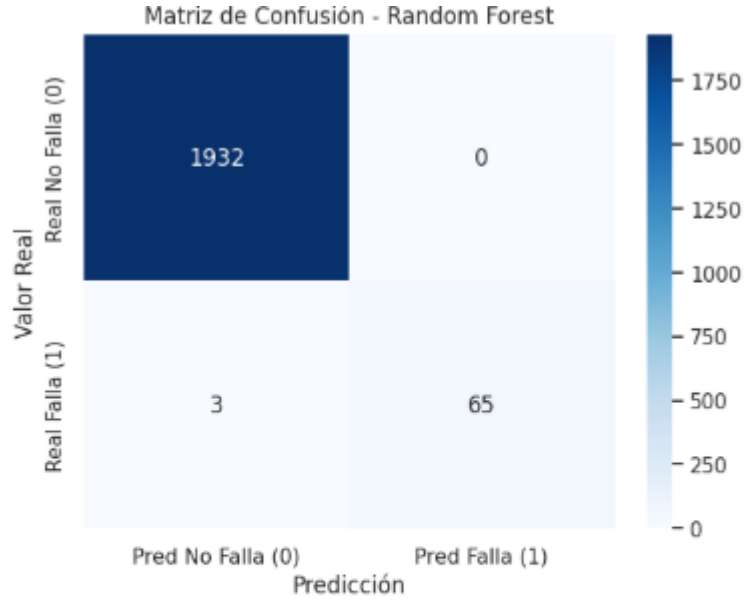


Figura 2. Correlación de sensores y ocurrencia de fallas



ANALISIS: Muestra la relación entre Torque y Velocidad Rotacional, donde los fallos (naranja) tienden a concentrarse en extremos de carga.




```
--- Reporte de Clasificación (Random Forest) ---
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1932
           1       1.00      0.96      0.98        68

    accuracy                           1.00      2000
   macro avg       1.00      0.98      0.99      2000
weighted avg       1.00      1.00      1.00      2000
```

Figura 3. Matriz de Confusión - Desempeño Random Forest.

ANALISIS: El modelo es altamente eficaz: solo hubo 3 fallos omitidos (Falsos Negativos) frente a 65 detectados exitosamente.

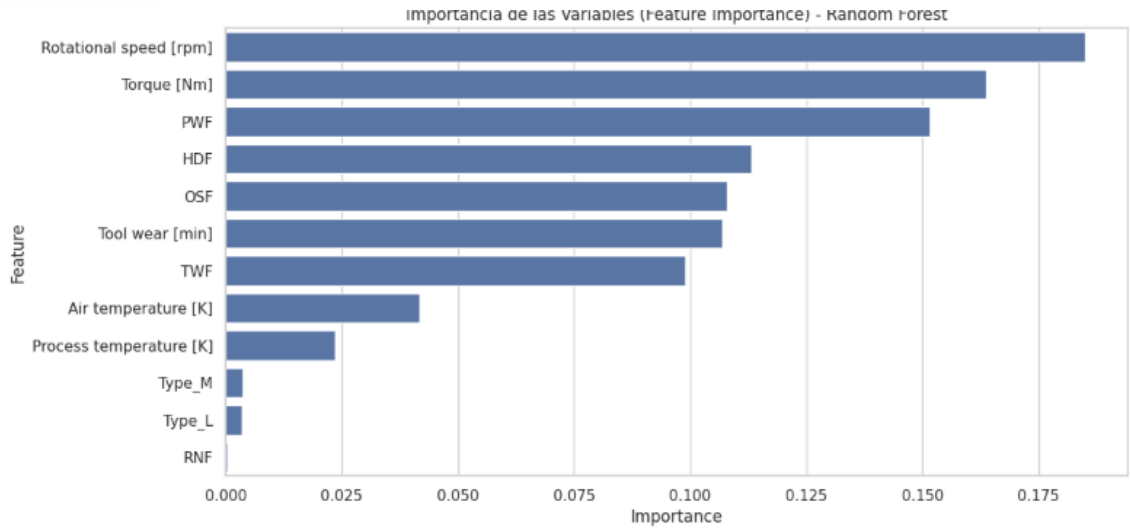


Figura 4. Importancia de las variables (Feature Importance).

ANALISIS: Se identifica que la Velocidad Rotacional y el Torque son los factores con mayor peso predictivo en la falla de la maquinaria.

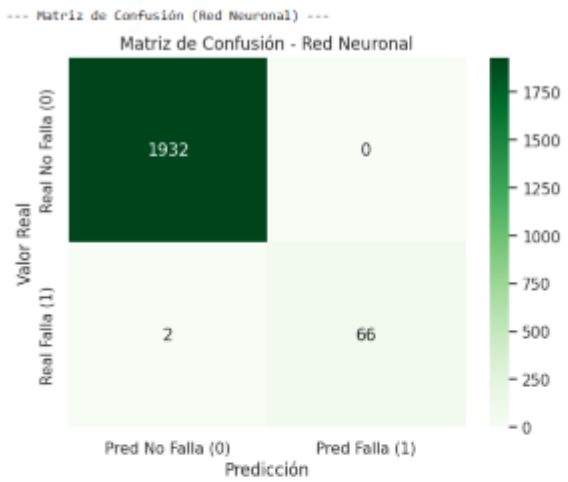
En este escenario, el Accuracy es una métrica engañosa debido a la naturaleza de los datos.

Línea base de predicción: Si un modelo predijera que ninguna máquina fallará jamás, obtendría un 96.6% de Accuracy simplemente por la proporción de la Clase 0. Por lo tanto, un 95% de exactitud es, de hecho, peor que un modelo que no ha aprendido nada.

Impacto operativo: La gestión de mantenimiento depende de la detección temprana. Un modelo con 95% de exactitud que falla en predecir el 5% de los eventos (que son justamente los fallos) resulta inútil para prevenir paradas no programadas o daños catastróficos.

Métricas críticas: Para este caso, es más importante observar la Matriz de Confusión y el Recall de la Clase 1. En los resultados obtenidos, el modelo Random Forest muestra un desempeño superior al lograr detectar 65 de 68 fallos, lo cual es el verdadero valor del sistema.

Interpretación de la Matriz de Confusión




```
print("\n--- Reporte de Clasificación (Red Neuronal) ---")
print(classification_report(y_test, y_pred_nn))
```

```
--- Reporte de Clasificación (Red Neuronal) ---
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1932
           1       1.00      0.97      0.99        68

    accuracy                           1.00      2000
   macro avg       1.00      0.99      0.99      2000
weighted avg       1.00      1.00      1.00      2000
```

Tarea de Cálculo de Métricas:

Verdaderos Positivos (TP): 66

Verdaderos Negativos (TN): 1932

Falsos Positivos (FP): 0

Falsos Negativos (FN): 2

Precisión: $\frac{66}{66+0}$: 1.00 (100%)

Interpretación: El modelo es extremadamente confiable; cuando emite una alerta de falla, siempre es real.

Sensibilidad: $\frac{66}{66+2}$: 0,97 (97%)

Interpretación: El modelo logra capturar el 97% de los fallos reales totales, dejando escapar solo 2 eventos.

Costo Total de Error: $\boldsymbol{Costo}: (\mathbf{2x5000}) + (\mathbf{0x200}): \boldsymbol{USD}\ \mathbf{10,000}$

Pregunta: ¿Preferiría un modelo con un Recall del 98% pero una Precision del 60%, o viceversa?

Respuesta: Preferiría el modelo con Recall del 98%.

Justificación: En manufactura, el costo de un fallo no detectado (FN = USD 5,000) es 25 veces mayor que el de una falsa alarma (FP = USD 200). Es preferible inspeccionar la máquina innecesariamente algunas veces (baja precisión) que permitir que la línea de producción se detenga por una avería catastrófica no detectada

Pregunta: ¿Son suficientes 30 épocas? ¿Qué sucede con el error de validación?

Respuesta: En problemas de mantenimiento predictivo con datasets de este tamaño, 30 épocas suelen ser suficientes para que el modelo converja si la curva de pérdida se estabiliza. Si el error de validación comienza a subir mientras el de entrenamiento baja, estaríamos ante un caso de sobreajuste, lo que indicaría que el modelo está memorizando el ruido en lugar de aprender los patrones de falla.

Pregunta: ¿Cómo el uso de métricas avanzadas transforma la rentabilidad?

Respuesta: El uso de métricas como el Recall permite pasar de un mantenimiento reactivo (reparar cuando se rompe) a uno proactivo. Al optimizar el Recall, la empresa minimiza los costos por paradas no planificadas, lo que impacta directamente en la rentabilidad al asegurar la continuidad operativa y extender la vida útil de los activos, algo que el simple "Accuracy" no puede reflejar en procesos críticos.